

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Алгоритми та методи обчислень

Лабораторна робота №4

«Розв’язання нелінійних рівнянь на комп’ютері»

Виконав: студент
групи ІО-42
Куліков М. М.
Номер у списку групи: 9
Перевірив:
Порєв В. М.

Лабораторна робота №4

Тема: «Розв’язання нелінійних рівнянь на комп’ютері».

Мета: Ознайомлення методиками та вивчення різних алгоритмів розв’язання нелінійних рівнянь на комп’ютері.

Загальне завдання:

- Ознайомитись з методикою розв’язання нелінійних рівнянь на ПК.
- Побудувати графік функції лівої частини рівняння.
- Розв’язати рівняння методом дотичних (Ньютона).
- Скласти схему алгоритму розв’язання нелінійного рівняння і на базі нього скласти програму розв’язання.
- Відокремити корені в рівнянні, що досліджується (Відокремлення коренів виконати на ПК, розробивши для цього програму).
- Вибрати значення точності обчислень ϵ .
- На кожному проміжку уточнити корені, користуючись розробленою програмою.
- Початкові дані (проміжок, точність обчислень ϵ) і результати роздрукувати.
- Використовуючи блок-схему алгоритму, написати програму розв’язування нелінійного рівняння мовою програмування, узгодженою з викладачем.
- Оформити звіт, що відповідає таким вимогам.

Індивідуальне завдання:

Завдання виконане згідно 9 варіанту.

Таблиця 1 – Завдання згідно варіанту

Метод	Номер варіанту	Рівняння	Примітка
Метод дотичних	9	$x^3 - x - 3 = 0$	1.213

Блок-схеми алгоритмів:

Загальна логіка роботи програми полягає у двох етапах: програмного відокремлення кореня методом табуляції на заданому відрізку та подальшого точного розв'язання рівняння. Основним обчислювальним ядром є метод дотичним, тому детальну блок-схему саме цього алгоритму розв'язання наведено на рисунку нижче

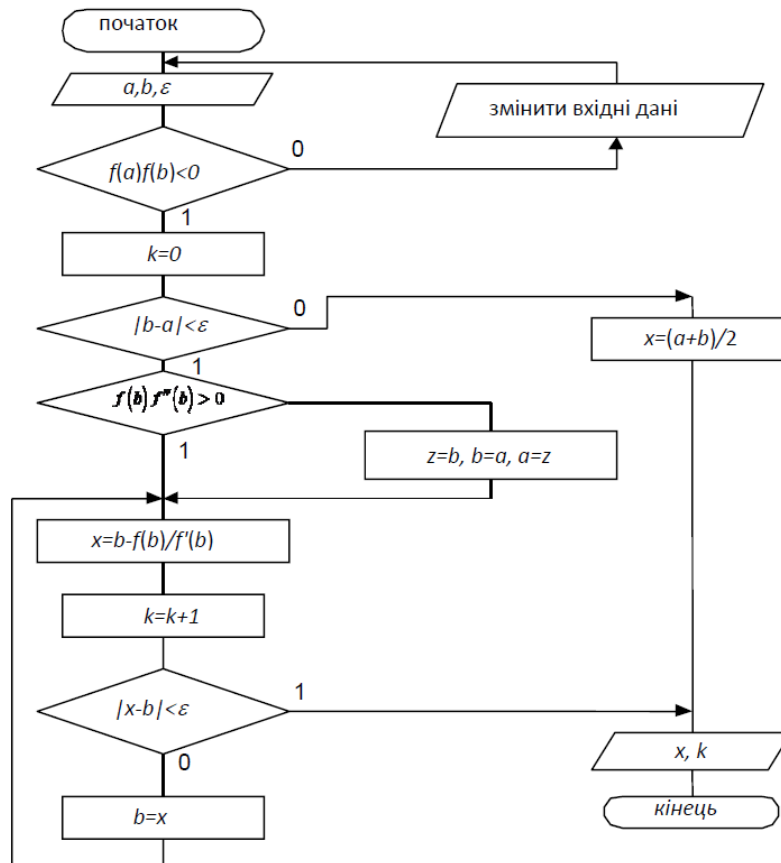


Рисунок 1 – Блок схема методу дотичних

Код програми:

Оскільки програма написана на С# з використанням WPF, і код є громіздким, то вміст усіх файлів представлено в додатку А. У цьому розділі наведений код для реалізації методу дотичних.

Вміст файлу TangentMath.cs:

```
internal class TangentMath
{
    public List<(double, double)> SeparateRoots(IEquation eq, double a, double b, double h)
    {
        var intervals = new List<(double, double)>();
        double x = a;

        while (x < b)
        {
            double nextX = Math.Min(x + h, b);
            double fx1 = eq.Evaluate(x);
            double fx2 = eq.Evaluate(nextX);
            if (fx1 * fx2 <= 0) intervals.Add((x, nextX));
            x = nextX;
        }
        return intervals;
    }

    public List<IterationStep> FindRoot(IEquation eq, double epsilon, out double root)
    {
        var steps = new List<IterationStep>();
        double x_n = eq.Evaluate(eq.A) * eq.EvaluateSecondDerivative(eq.A) > 0 ? eq.A : eq.B;
        int iteration = 0;
        double delta = double.MaxValue;
        while (delta > epsilon && iteration < 1000)
        {
            double f_xn = eq.Evaluate(x_n);
            double df_xn = eq.EvaluateFirstDerivative(x_n);
            if (Math.Abs(df_xn) < 1e-15) break;
            double x_next = x_n - f_xn / df_xn;
            delta = Math.Abs(x_next - x_n);
            steps.Add(new IterationStep { N = iteration, Xn = x_n, F_Xn = f_xn, Df_Xn = df_xn, X_Next = x_next, Delta = delta });
            x_n = x_next;
            iteration++;
        }

        root = x_n;
        return steps; } }
```

Результати виконання програми:

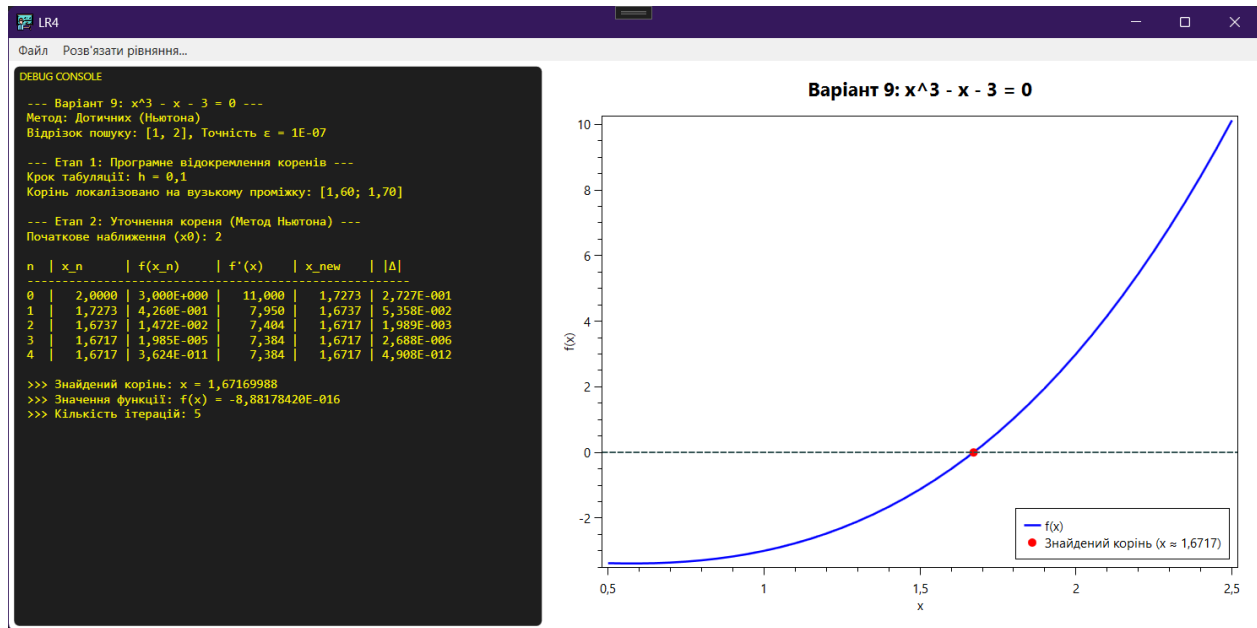


Рисунок 2 – Результат запуску розв’язання нелінійного рівняння $x^3 - x - 3 = 0$

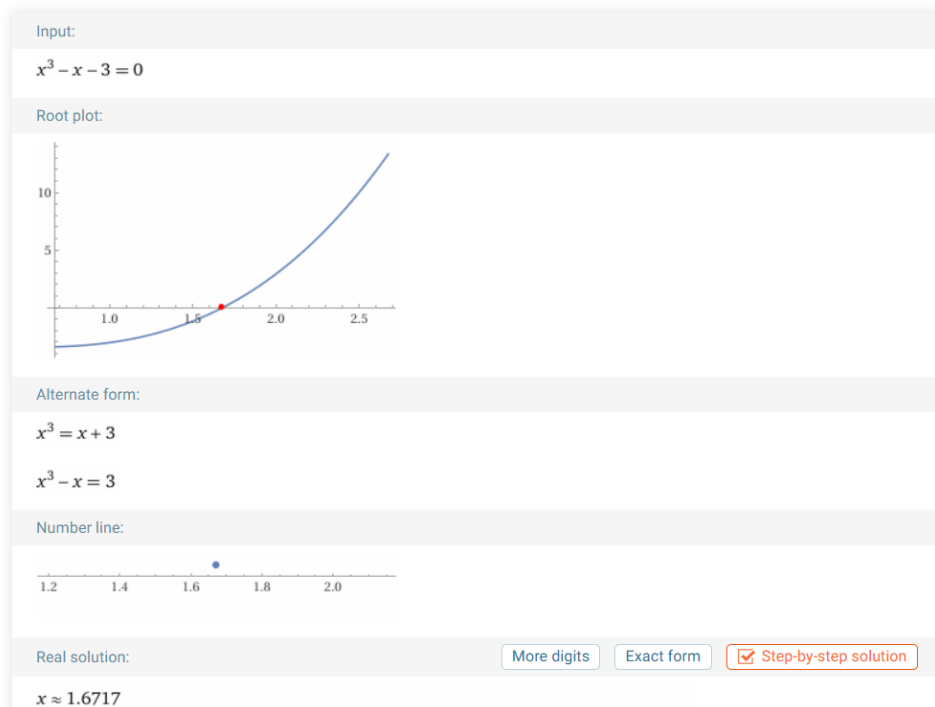


Рисунок 3 – Результат обчислень на сайті Wolfram Alpha

Висновки:

Отже, під час виконання лабораторної роботи було створено програму мовою C# з використанням технології WPF, яка реалізує чисельне розв'язання нелінійного рівняння $x^3 - x - 3 = 0$ методом дотичних (Ньютона). Програма успішно обрахувала корінь $x \approx 1,6717$ на відрізку [1,2] всього за 4 ітерації із заданою точністю $\epsilon = 10^{-7}$. Побудований графічний інтерфейс із візуалізацією функції підтвердив правильність розрахунків, оскільки знайдений корінь точно співпадає з точкою перетину графіка з віссю абсцис.

Додаток А

Вміст файлу MainWindow.xaml:

```
<Window x:Class="LR4.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:local="clr-namespace:LR4" xmlns:oxy="http://oxyplot.org/wpf"
    mc:Ignorable="d"
    Title="LR4" Height="600" Width="1200" WindowStartupLocation="CenterScreen" Icon="/icons8-mathematics-lineal-color-96.png">

    <DockPanel>
        <Menu DockPanel.Dock="Top" Padding="2">
            <MenuItem Header="Файл">
                <MenuItem Header="Зберегти звіт" Click="SaveToFile"/>
                <MenuItem Header="Вихід" Click="ExitApp"/>
            </MenuItem>
            <MenuItem Header="Розв'язати рівняння...">
                <MenuItem Header="Варіант 9:  $x^3 - x - 3 = 0$ " Click="RunVariantFunction"/>
            </MenuItem>
        </Menu>

        <Grid Margin="5">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="500"/>
                <ColumnDefinition Width="*" />
            </Grid.ColumnDefinitions>

            <Border Grid.Column="0" Background="#1E1E1E" CornerRadius="5">
                <DockPanel>
                    <TextBlock DockPanel.Dock="Top" Text="DEBUG  CONSOLE" Foreground="#FFFFFF906" FontSize="10"
                        Margin="5,2"/>
                    <TextBox x:Name="txtLog" Background="Transparent" Foreground="#FFFFFF906"
                        BorderThickness="0" VerticalScrollBarVisibility="Auto" IsReadOnly="True"
                        FontFamily="Consolas" FontSize="12" Padding="10" TextWrapping="Wrap"/>
                </DockPanel>
            </Border>

            <oxy:PlotView x:Name="plotView" Grid.Column="1" Background="White" Margin="5,0,0,0"/>
        </Grid>
    </DockPanel>
</Window>
```

Вміст файлу MainWindow.xaml.cs:

```
using LR4.Models;
using LR4.Models.Equations;
using System.Windows;

namespace LR4
```

```

{
    public partial class MainWindow : Window
    {
        private readonly TangentSolver _solver;
        public MainWindow()
        {
            InitializeComponent();
            _solver = new TangentSolver();
        }

        private void RunVariantFunction(object sender, RoutedEventArgs e)
        {
            var paramWin = new ParametersWindow("1", "2", "0,0001") { Owner = this };
            if (paramWin.ShowDialog() == true)
            {
                var eq = new CustomEquation(paramWin.A, paramWin.B);
                ExecuteAnalysis(eq, paramWin.Epsilon);
            }
        }

        private void ExecuteAnalysis(IEquation eq, double epsilon)
        {
            txtLog.Clear();
            AnalysisResult result = _solver.Solve(eq, epsilon);
            plotView.Model = result.GraphModel;
            plotView.InvalidatePlot(true);
            foreach (var line in result.LogLines) { txtLog.AppendText(line + "\n"); }
            txtLog.ScrollToEnd();
        }

        private void SaveToFile(object sender, RoutedEventArgs e)
        {
            var dialog = new Microsoft.Win32.SaveFileDialog { Filter = "Text Files (*.txt)|*.txt", FileName = "Report_LR4.txt" };
            if (dialog.ShowDialog() == true) {
                try {
                    DataHelper.SaveLog(dialog.FileName, txtLog.Text);
                    MessageBox.Show("Звіт успішно збережено!");
                }
                catch (Exception ex) { MessageBox.Show(ex.Message, "Помилка", MessageBoxButton.OK, MessageBoxImage.Error); }
            }
        }

        private void ExitApp(object sender, RoutedEventArgs e) { Application.Current.Shutdown(); }
    }
}

```

Вміст файлу ParametersWindow.xaml:

```

<Window x:Class="LR4.ParametersWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"

```

```

xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:LR4"
mc:Ignorable="d"
Title="Налаштування параметрів" Height="250" Width="300" WindowStartupLocation="CenterOwner"
ResizeMode="NoResize">
<StackPanel Margin="15">
    <TextBlock Text="Ліва межа (A):" Margin="0,0,0,5"/>
    <TextBox x:Name="txtA" Text="1" Margin="0,0,0,10" Padding="2"/>

    <TextBlock Text="Права межа (B):" Margin="0,0,0,5"/>
    <TextBox x:Name="txtB" Text="2" Margin="0,0,0,10" Padding="2"/>

    <TextBlock Text="Точність (ε):" Margin="0,0,0,5"/>
    <TextBox x:Name="txtEpsilon" Text="0,0001" Margin="0,0,0,15" Padding="2"/>

    <Button Content="Обчислити" Click="BtnCompute_Click" Padding="5" IsDefault="True"/>
</StackPanel>
</Window>

```

Вміст файлу ParametersWindow.xaml.cs:

```

using System.Windows;

namespace LR4
{
    public partial class ParametersWindow : Window
    {
        public double A { get; private set; }
        public double B { get; private set; }
        public double Epsilon { get; private set; }

        public ParametersWindow(string defaultA, string defaultB, string defaultEps)
        {
            InitializeComponent();
            txtA.Text = defaultA;
            txtB.Text = defaultB;
            txtEpsilon.Text = defaultEps;
        }

        private void BtnCompute_Click(object sender, RoutedEventArgs e)
        {
            if (double.TryParse(txtA.Text.Replace('.', ','), out double a) &&
                double.TryParse(txtB.Text.Replace('.', ','), out double b) &&
                double.TryParse(txtEpsilon.Text.Replace('.', ','), out double eps))
            {
                if (a >= b)
                {
                    MessageBox.Show("Межа A повинна бути меншою за B!", "Помилка", MessageBoxButton.OK,
                        MessageBoxImage.Warning);
                    return;
                }
            }
        }
    }
}

```

```

        if (eps <= 0)
        {
            MessageBox.Show("Точність повинна бути додатним числом!", "Помилка", MessageBoxButton.OK,
            MessageBoxImage.Warning);
            return;
        }
        A = a;
        B = b;
        Epsilon = eps;
        DialogResult = true;
    }
    else { MessageBox.Show("Некоректний формат чисел!", "Помилка", MessageBoxButton.OK, MessageBoxImage.Error); }
} } }

```

Вміст файлу IEquation.cs:

```

namespace LR4.Models.Equations
{
    public interface IEquation
    {
        string Title { get; }
        double A { get; }
        double B { get; }
        double Evaluate(double x);
        double EvaluateFirstDerivative(double x);
        double EvaluateSecondDerivative(double x);
    }
}

```

Вміст файлу CustomEquation.cs:

```

namespace LR4.Models.Equations
{
    internal class CustomEquation: IEquation
    {
        public string Title => "Варіант 9:  $x^3 - x - 3 = 0$ ";
        public double A { get; }
        public double B { get; }

        public CustomEquation(double a, double b)
        {
            A = a;
            B = b;
        }

        public double Evaluate(double x) => Math.Pow(x, 3) - x - 3.0;
        public double EvaluateFirstDerivative(double x) => 3.0 * Math.Pow(x, 2) - 1.0;
        public double EvaluateSecondDerivative(double x) => 6.0 * x;
    }
}

```

Вміст файлу AnalysisResult.cs:

```

using OxyPlot;
namespace LR4.Models

```

```
{
    public class AnalysisResult
    {
        public PlotModel GraphModel { get; set; }
        public List<string> LogLines { get; set; } = new List<string>();
    }
}
```

Вміст файлу DataHelper.cs:

```
using System.IO;
namespace LR4.Models
{
    public static class DataHelper
    {
        public static void SaveLog(string path, string content)
        {
            if (string.IsNullOrEmpty(content)) throw new ArgumentException("Немає даних для збереження.");
            File.WriteAllText(path, content);
        }
    }
}
```

Вміст файлу IterationStep.cs:

```
namespace LR4.Models
{
    public class IterationStep
    {
        public int N { get; set; }
        public double Xn { get; set; }
        public double F_Xn { get; set; }
        public double Df_Xn { get; set; }
        public double X_Next { get; set; }
        public double Delta { get; set; }
    }
}
```

Вміст файлу TangentMath.cs:

```
using LR4.Models.Equations;
namespace LR4.Models
{
    internal class TangentMath
    {
        public List<(double, double)> SeparateRoots(IEquation eq, double a, double b, double h)
        {
            var intervals = new List<(double, double)>();
            double x = a;

            while (x < b)
            {
                double nextX = Math.Min(x + h, b);
                double fx1 = eq.Evaluate(x);
                double fx2 = eq.Evaluate(nextX);
                if (fx1 * fx2 <= 0) intervals.Add((x, nextX));
            }
        }
    }
}
```

```

        x = nextX;
    }
    return intervals;
}

public List<IterationStep> FindRoot(IEquation eq, double epsilon, out double root)
{
    var steps = new List<IterationStep>();
    double x_n = eq.Evaluate(eq.A) * eq.EvaluateSecondDerivative(eq.A) > 0 ? eq.A : eq.B;
    int iteration = 0;
    double delta = double.MaxValue;

    while (delta > epsilon && iteration < 1000)
    {
        double f_xn = eq.Evaluate(x_n);
        double df_xn = eq.EvaluateFirstDerivative(x_n);
        if (Math.Abs(df_xn) < 1e-15) break;
        double x_next = x_n - f_xn / df_xn;
        delta = Math.Abs(x_next - x_n);
        steps.Add(new IterationStep { N = iteration, Xn = x_n, F_Xn = f_xn, Df_Xn = df_xn, X_Next = x_next, Delta = delta });
        x_n = x_next;
        iteration++;
    }
    root = x_n;
    return steps;
} } }

```

Вміст файлу TangentSolver.cs:

```

using LR4.Models.Equations;
using OxyPlot;
using OxyPlot.Axes;
using OxyPlot.Series;

namespace LR4.Models
{
    public class TangentSolver
    {
        public AnalysisResult Solve(IEquation eq, double epsilon)
        {
            var result = new AnalysisResult();
            result.LogLines.Add($"--- {eq.Title} ---");
            result.LogLines.Add($"Метод: Дотичних (Ньютона)");
            result.LogLines.Add($"Відрізок пошуку: [{eq.A}, {eq.B}], Точність  $\epsilon = {epsilon}\backslash n$ ");
            var calculator = new TangentMath();
            result.LogLines.Add($"Етап 1: Програмне відокремлення коренів ---");
            double step = 0.1;
            var isolatedIntervals = calculator.SeparateRoots(eq, eq.A, eq.B, step);

            if (isolatedIntervals.Count == 0)
            {

```

```

        result.LogLines.Add($"[!] На відріжку [{eq.A}; {eq.B}] коренів не виявлено.");
        return result;
    }

    result.LogLines.Add($"Крок табуляції: h = {step}");
    foreach (var interval in isolatedIntervals)
    {
        result.LogLines.Add($"Корінь локалізовано на вузькому проміжку: [{interval.Item1:F2}; {interval.Item2:F2}]");
    }
    result.LogLines.Add("");
    result.LogLines.Add("--- Етап 2: Уточнення кореня (Метод Ньютона) ---");
    var steps = calculator.FindRoot(eq, epsilon, out double finalRoot);

    if (steps.Count > 0) result.LogLines.Add($"Початкове наближення (x0): {steps[0].Xn}\n");
    result.LogLines.Add($"{"n",-2} | {"x_n",-8} | {"f(x_n",-10} | {"f'(x)",-8} | {"x_new",-8} | {"|Δ|",-9}");
    result.LogLines.Add(new string('-', 55));

    foreach (var _step in steps)
    {
        result.LogLines.Add($"{"_step.N",-2} | {"_step.Xn,8:F4} | {"_step.F_Xn,10:E3} | {"_step.Df_Xn,8:F3} | {"_step.X_Next,8:F4} | {"_step.Delta,9:E3}");
    }

    result.LogLines.Add($"n>>> Знайдений корінь: x = {finalRoot:F8}");
    result.LogLines.Add($">>> Значення функції: f(x) = {eq.Evaluate(finalRoot):E8}");
    result.LogLines.Add($">>> Кількість ітерацій: {steps.Count}");
    var plotModel = new PlotModel { Title = eq.Title, Background = OxyColors.White };
    plotModel.Axes.Add(new LinearAxis { Position = AxisPosition.Bottom, Title = "x" });
    plotModel.Axes.Add(new LinearAxis { Position = AxisPosition.Left, Title = "f(x)" });
    plotModel.Legends.Add(new OxyPlot.Legends.Legend { LegendPosition = OxyPlot.Legends.LegendPosition.BottomRight,
        LegendBackground = OxyColor.FromAColor(220, OxyColors.White), LegendBorder = OxyColors.Black });
    plotModel.Annotations.Add(new OxyPlot.Annotations.LineAnnotation { Type = OxyPlot.Annotations.LineAnnotationType.Horizontal, Y = 0, Color = OxyColors.DarkSlateGray, LineStyle = LineStyle.Dash, StrokeThickness = 1.5 });
    plotModel.Annotations.Add(new OxyPlot.Annotations.LineAnnotation { Type = OxyPlot.Annotations.LineAnnotationType.Vertical, X = 0, Color = OxyColors.DarkSlateGray, LineStyle = LineStyle.Dash, StrokeThickness = 1.5 });
    var funcSeries = new LineSeries { Title = "f(x)", Color = OxyColors.Blue, StrokeThickness = 2, TrackerFormatString = "{0}\nx: {2:F4}\ny: {4:F4}" };
    for (double x = eq.A - 0.5; x <= eq.B + 0.5; x += 0.05)
    { funcSeries.Points.Add(new DataPoint(x, eq.Evaluate(x))); }
    plotModel.Series.Add(funcSeries);

    var rootSeries = new ScatterSeries { Title = $"Знайдений корінь (x ≈ {finalRoot:F4})", MarkerType = MarkerType.Circle,
        MarkerFill = OxyColors.Red, MarkerSize = 4, TrackerFormatString = "{0}\nx: {2:F4}\ny: {4:F4}" };
    rootSeries.Points.Add(new ScatterPoint(finalRoot, 0));
    plotModel.Series.Add(rootSeries);
    result.GraphModel = plotModel;
    return result;
} }

```